

Python for Computer Science and Data Science 2 (CSE 3652)

MINOR ASSIGNMENT-1: OBJECT-ORIENTED PROGRAMMING (OOP)

Card Game Simulation and Analysis

1 Problem Statement

Design and implement a basic card game framework that supports simple card games like Blackjack. The framework should include a deck of cards, players, and basic game rules. The goal of this project is to create an extendable and reusable structure that can accommodate the card game by implementing fundamental components such as a deck of cards, a hand, and a set of game rules.

For example, in Blackjack, the objective is to achieve a hand value as close to 21 as possible without exceeding it. The game involves a dealer and one or more players. The dealer follows fixed rules for drawing cards, while players can decide whether to draw additional cards.

2 Tasks

2.1 Task 1: Create Card and Deck Classes

- Create a `Card` class that represents a single playing card. Each card should have a **suit** (Hearts, Diamonds, Clubs, or Spades) and a **rank** (Ace, 2, 3, ..., 10, Jack, Queen, King).
- Use an `enum` type to define suits instead of using string values. This ensures consistency and prevents errors due to typos.
- Implement a `Deck` class that generates a full deck of 52 cards, consisting of four suits and thirteen ranks each.
- Add functionality to shuffle the deck randomly.
- Implement a method to draw a card from the deck, removing it from the available cards.
- **Example Data:**
 - Example of a single card: `Card(suit=Hearts, rank=Ace)`.
 - Example of a shuffled deck: `[Card(Spades, King), Card(Diamonds, 7), Card(Hearts, 2), ...]`.

```
In [1]: # =====  
# TASK 1: CARD AND DECK CLASSES  
# =====  
  
# Import required modules  
import random  
from enum import Enum  
  
# -----  
# Create Suit Enum  
# -----  
# Enum is used to avoid spelling mistakes  
# and maintain fixed suit values.  
class Suit(Enum):  
    HEARTS = "Hearts"  
    DIAMONDS = "Diamonds"  
    CLUBS = "Clubs"  
    SPADES = "Spades"
```

```
# -----  
# Create Card Class  
# -----  
# Represents a single playing card.  
class Card:  
  
    # Constructor to initialize suit and rank  
    def __init__(self, suit, rank):  
        self.suit = suit  
        self.rank = rank  
  
    # String representation of card  
    def __str__(self):  
        return f"{self.rank} of {self.suit.value}"  
  
# -----  
# Create Deck Class  
# -----  
# Represents a complete deck of 52 cards.  
class Deck:  
  
    # Constructor  
    def __init__(self):  
        self.cards = []  
        self.create_deck()  
    # Method to create all 52 cards  
    def create_deck(self):  
        # List of all ranks  
        ranks = [  
            "Ace", "2", "3", "4", "5", "6",  
            "7", "8", "9", "10",  
            "Jack", "Queen", "King"  
        ]  
        # Generate cards for every suit and rank  
        for suit in Suit:  
            for rank in ranks:  
                card = Card(suit, rank)  
                self.cards.append(card)  
    # Method to shuffle cards randomly  
    def shuffle(self):  
        random.shuffle(self.cards)  
  
    # Method to draw one card from deck  
    def draw_card(self):  
  
        # pop() removes the last card  
        return self.cards.pop()  
  
# -----  
# Testing Task 1  
# -----  
  
# Create deck object  
deck = Deck()  
# Shuffle the deck  
deck.shuffle()  
  
# Draw 5 cards  
print("5 Drawn Cards:\n")  
for i in range(5):  
    card = deck.draw_card()  
    print(card)
```

5 Drawn Cards:

6 of Clubs
9 of Clubs
King of Clubs
Queen of Diamonds
10 of Clubs

2.2 Task 2: Implement a Simple Blackjack Game

- Blackjack is a game where players aim to get as close to 21 points as possible without exceeding it. Face cards (King, Queen, Jack) are worth 10 points, Aces can be worth 1 or 11 points, and other cards retain their numeric values.
- The dealer and the player are both dealt two cards at the start of the game.
- The player can choose to **hit** (draw an additional card) or **stand** (don't take any more cards.).
- If the player's total exceeds 21, they go **bust** and lose the game.
- The dealer must follow a set rule: if their total is 16 or below, they must hit; if 17 or above, they must stand.
- The player wins if their total is higher than the dealer's without exceeding 21.
- **Example Scenario:**
 - **Player's Hand:** [Card(Hearts, 10), Card(Clubs, 7)]
Total: 17
 - **Dealer's Hand:** [Card(Spades, 8), Card(Diamonds, 10)]
Total: 18
 - **Result:** Dealer wins.

```
In [2]: # =====  
# TASK 2: SIMPLE BLACKJACK GAME  
# =====  
  
# Import required modules  
import random  
from enum import Enum  
  
# -----  
# Suit Enum  
# -----  
# Enum is used to define fixed suit values.  
class Suit(Enum):  
    HEARTS = "Hearts"  
    DIAMONDS = "Diamonds"  
    CLUBS = "Clubs"  
    SPADES = "Spades"  
  
# -----  
# Card Class  
# -----  
# Represents a single card.  
class Card:  
  
    # Constructor  
    def __init__(self, suit, rank):  
        self.suit = suit  
        self.rank = rank
```

```
# Returns card value
def value(self):

    # Face cards value = 10
    if self.rank in ["Jack", "Queen", "King"]:
        return 10

    # Ace value = 11 initially
    elif self.rank == "Ace":
        return 11

    # Numeric cards
    else:
        return int(self.rank)

# Display card
def __str__(self):
    return f"{self.rank} of {self.suit,value}"

# -----
# Deck Class
# -----
# Represents full deck of cards.
class Deck:

    # Constructor
    def __init__(self):
        self.cards = []
        self.create_deck()

    # Create 52 cards
    def create_deck(self):

        ranks = [
            "Ace", "2", "3", "4", "5", "6",
            "7", "8", "9", "10",
            "Jack", "Queen", "King"
        ]

        # Generate all cards
        for suit in Suit:
            for rank in ranks:
                self.cards.append(Card(suit, rank))

    # Shuffle deck
    def shuffle(self):
        random.shuffle(self.cards)

    # Draw one card
    def draw_card(self):
        return self.cards.pop()

# -----
# Player Class
# -----
# Represents player or dealer.
class Player:

    # Constructor
    def __init__(self, name):
        self.name = name
        self.hand = []

    # Add card to hand
    def add_card(self, card):
        self.hand.append(card)

    # Calculate total hand value
```

```
def hand_value(self):
    total = 0
    aces = 0
    for card in self.hand:
        total += card.value()
        # Count aces
        if card.rank == "Ace":
            aces += 1

    # Convert Ace from 11 to 1 if total > 21
    while total > 21 and aces > 0:
        total -= 10
        aces -= 1
    return total

# Check bust condition
def is_bust(self):
    return self.hand_value() > 21

# Display player's hand
def show_hand(self):
    print(f"\n{self.name}'s Hand:")
    for card in self.hand:
        print(card)
    print("Total:", self.hand_value())

# -----
# Blackjack Game Class
# -----
class BlackjackGame:

    # Constructor
    def __init__(self):

        # Create deck
        self.deck = Deck()

        # Shuffle deck
        self.deck.shuffle()

        # Create player
        self.player = Player("Player")

        # Create dealer
        self.dealer = Player("Dealer")

    # Give initial 2 cards
    def initial_deal(self):

        for i in range(2):
            self.player.add_card(self.deck.draw_card())
            self.dealer.add_card(self.deck.draw_card())

    # Player turn
    def player_turn(self):
        while True:

            # Show player's cards
            self.player.show_hand()

            # Check if player busted
            if self.player.is_bust():
                print("\nPlayer Busted!")
                return

            # Take input
            choice = input("\nHit or Stand? (H/S): ")
```

```
# HIT
if choice == 'H':
    self.player.add_card(self.deck.draw_card())

# STAND
elif choice == 'S':
    break

# Invalid input
else:
    print("Invalid Input! Please enter H or S.")

# Dealer turn
def dealer_turn(self):
    print("\n===== DEALER TURN =====")

    # Dealer hits on 16 or below
    while self.dealer.hand_value() <= 16:
        self.dealer.add_card(self.deck.draw_card())

    # Show dealer cards
    self.dealer.show_hand()

# Decide winner
def decide_winner(self):
    player_total = self.player.hand_value()
    dealer_total = self.dealer.hand_value()
    print("\n===== RESULT =====")

    # Player busted
    if player_total > 21:
        print("Dealer Wins!")

    # Dealer busted
    elif dealer_total > 21:
        print("Player Wins!")

    # Compare scores
    elif player_total > dealer_total:
        print("Player Wins!")
    elif dealer_total > player_total:
        print("Dealer Wins!")
    else:
        print("Match Tie!")

# Start game
def play_game(self):
    # Initial cards
    self.initial_deal()
    # Player turn
    self.player_turn()
    # Dealer turn only if player not busted
    if not self.player.is_bust():
        self.dealer_turn()
    # Final result
    self.decide_winner()

# -----
# Main Program
# -----

print("===== BLACKJACK GAME =====")

# Create game object
game = BlackjackGame()
# Start game
game.play_game()
```

===== BLACKJACK GAME =====

Player's Hand:
Jack of Diamonds
2 of Diamonds
Total: 12

Player's Hand:
Jack of Diamonds
2 of Diamonds
8 of Spades
Total: 20

Player's Hand:
Jack of Diamonds
2 of Diamonds
8 of Spades
2 of Hearts
Total: 22

Player Busted!

===== RESULT =====
Dealer Wins!

2.3 Task 3: Implement Multiplayer Support

- Extend the framework to support multiple players in a game.
- Allow each player to take turns making decisions (e.g., hit or stand in Blackjack).
- Implement a turn-based system to manage player actions and game progression.
- Display player hands and game status after each round.
- **Example Data:**
 - Player 1 hand: [Card(Hearts, 7), Card(Spades, 5)].
 - Player 2 hand: [Card(Diamonds, 9), Card(Clubs, 6)].
 - Player 1 hits and receives Card(Hearts, 3).
 - Updated hand: [Card(Hearts, 7), Card(Spades, 5), Card(Hearts, 3)] (Total = 15).

```
In [3]: # =====  
# TASK 3: MULTIPLAYER BLACKJACK GAME  
# =====  
  
import random  
from enum import Enum  
  
# -----  
# Suit Enum  
# -----  
class Suit(Enum):  
    HEARTS = "Hearts"  
    DIAMONDS = "Diamonds"  
    CLUBS = "Clubs"  
    SPADES = "Spades"  
  
# -----  
# Card Class  
# -----
```

```
class Card:

    def __init__(self, suit, rank):
        self.suit = suit
        self.rank = rank

    # Return card value
    def value(self):

        # Face cards = 10
        if self.rank in ["Jack", "Queen", "King"]:
            return 10

        # Ace = 11 initially
        elif self.rank == "Ace":
            return 11

        # Number cards
        else:
            return int(self.rank)

    # Display card
    def __str__(self):
        return f"{self.rank} of {self.suit.value}"

# -----
# Deck Class
# -----
class Deck:
    def __init__(self):
        self.cards = []
        self.create_deck()
    # Create all 52 cards
    def create_deck(self):
        ranks = [
            "Ace", "2", "3", "4", "5", "6",
            "7", "8", "9", "10",
            "Jack", "Queen", "King"
        ]
        for suit in Suit:
            for rank in ranks:
                self.cards.append(Card(suit, rank))
    # Shuffle cards
    def shuffle(self):
        random.shuffle(self.cards)

    # Draw one card
    def draw_card(self):
        return self.cards.pop()

# -----
# Player Class
# -----
class Player:

    def __init__(self, name):
        self.name = name
        self.hand = []
        self.stand = False

    # Add card to hand
    def add_card(self, card):
        self.hand.append(card)
```

```
# Calculate hand value
def hand_value(self):
    total = 0
    aces = 0

    for card in self.hand:
        total += card.value()
        # Count aces
        if card.rank == "Ace":
            aces += 1
    # Adjust Ace value if total > 21
    while total > 21 and aces > 0:
        total -= 10
        aces -= 1
    return total
# Check bust condition
def is_bust(self):
    return self.hand_value() > 21

# Show player's hand
def show_hand(self):
    print(f"\n{self.name}'s Hand:")
    for card in self.hand:
        print(card)
    print("Total:", self.hand_value())
# -----
# Multiplayer Blackjack Game
# -----
class MultiplayerBlackjack:
    def __init__(self, player_names):

        # Create deck
        self.deck = Deck()
        # Shuffle deck
        self.deck.shuffle()
        # Create players list
        self.players = []

        for name in player_names:
            self.players.append(Player(name))
        # Create dealer
        self.dealer = Player("Dealer")

    # Give initial cards
    def initial_deal(self):
        for i in range(2):
            for player in self.players:
                player.add_card(self.deck.draw_card())
            self.dealer.add_card(self.deck.draw_card())
    # Player turn
    def player_turn(self, player):
        while not player.stand and not player.is_bust():

            # Show cards
            player.show_hand()

            # Take input
            choice = input(f"\n{player.name}: Hit or Stand? (H/S): ")

            # HIT
            if choice == 'H':
```

```
        player.add_card(self.deck.draw_card())

    # STAND
    elif choice == 'S':
        player.stand = True

    # Invalid input
    else:
        print("Invalid Input! Please enter H or S.")

    # Bust message
    if player.is_bust():
        print(f"{player.name} Busted!")

    # Dealer turn
    def dealer_turn(self):
        print("\n===== DEALER TURN =====")

        # Dealer hits on 16 or below
        while self.dealer.hand_value() <= 16:
            self.dealer.add_card(self.deck.draw_card())

        # Show dealer hand
        self.dealer.show_hand()

    # Final result
    def show_results(self):
        dealer_total = self.dealer.hand_value()
        print("\n===== FINAL RESULTS =====")
        for player in self.players:
            player_total = player.hand_value()
            print(f"\n{player.name}: {player_total}")
            print(f"Dealer: {dealer_total}")
            # Player bust
            if player.is_bust():
                print("Result: Lose")
            # Dealer bust
            elif self.dealer.is_bust():
                print("Result: Win")
            # Compare totals
            elif player_total > dealer_total:
                print("Result: Win")
            elif dealer_total > player_total:
                print("Result: Lose")
            else:
                print("Result: Tie")

    # Start game
    def play_game(self):
        # Initial cards
        self.initial_deal()
        # Each player's turn
        for player in self.players:
            self.player_turn(player)

        # Dealer turn
        self.dealer_turn()
        # Show results
        self.show_results()

# -----
# Main Program
# -----
```

```
print("===== MULTIPLAYER BLACKJACK =====")
while True:
    try:
        num_players = int(input("Enter number of players: "))
        if num_players > 0:
            break
        else:
            print("Please enter a positive number.")
    except ValueError:
        print("Invalid input! Enter numbers only.")
# Store player names
player_names = []
# Input player names
for i in range(num_players):
    name = input(f"Enter Player {i+1} Name: ")
    player_names.append(name)
# Create game object
game = MultiplayerBlackjack(player_names)
# Start game
game.play_game()
```

===== MULTIPLAYER BLACKJACK =====

Amulya's Hand:
8 of Hearts
6 of Spades
Total: 14

Amulya's Hand:
8 of Hearts
6 of Spades
5 of Clubs
Total: 19
Amulya Busted!

Kaushal's Hand:
5 of Spades
Queen of Diamonds
Total: 15

Kaushal's Hand:
5 of Spades
Queen of Diamonds
6 of Diamonds
Total: 21
Kaushal Busted!

===== DEALER TURN =====

Dealer's Hand:
Jack of Diamonds
6 of Hearts
8 of Spades
Total: 24

===== FINAL RESULTS =====

Amulya: 29
Dealer: 24
Result: Lose

Kaushal: 31
Dealer: 24
Result: Lose

NB:

Basic Rules of the Game:

- The goal is to get as close to 21 as possible without going over.
- Face cards (King, Queen, Jack) = **10 points**.
- Aces (A) = **1 or 11 points**, depending on what benefits the player.
- If your total is higher than the dealer's without busting (going over 21), **you win**.
- If you go over 21, you automatically **lose (bust)**.
- The dealer must **hit if they have 16 or less** and **stand on 17 or more**.

When Does a Blackjack Game End?

A game of Blackjack concludes in the following scenarios:

1. Immediate Loss: Player Busts

- If a player's total exceeds **21**, they automatically lose, and their participation in the game ends.
- Other players continue until they either bust or stand.

2. Immediate Win: Dealer Busts

- If the dealer's total exceeds **21**, all remaining players who have not busted automatically **win**.
- The game ends at this point.

3. All Players Have Stood

- Once every player has either **busted** or **stood**, the dealer plays their turn.
- The dealer must:
 - **Hit on 16 or less.**
 - **Stand on 17 or more.**
- The game proceeds to the final comparison when the dealer stands or busts.

4. Final Comparison (Game Ends)

- The dealer's total is compared with each standing player:
 - If the **player's total is higher than the dealer's**, the player **wins**.
 - If the **dealer's total is higher**, the player **loses**.
 - If both have the **same total**, it's a **push (tie)**, and the player's bet is returned.
- After this comparison, the game ends.

Submitted By-

- Name- Amulya Shrivastava
- Regd No- 2341019456
- Section- 23412F3
- Serial No- 59
- Subject- PFCD2 Major Assignment-01